

Chapter 47

LP I2S Controller

47.1 Introduction

ESP32-P4 has a built-in LP I2S interface, which provides a data reception communication interface for [Voice Activity Detection \(VAD\)](#) and some digital audio applications in low power mode.

The I2S standard bus defines three signals, namely, a bit clock signal (BCK), a channel/word select signal (WS), and a serial data signal (SD). A basic I2S data bus has one master and one slave. The roles remain unchanged throughout the communication.

The LP I2S module on ESP32-P4 provides an independent RX unit, which supports receiving data when the chip is running with the lowest power consumption. Compared to HP I2S, i.e., I2S_O, I2S₁, and I2S₂, LP I2S does not support DMA access. Instead, it uses a separate internal memory to store data.

Note:

For I2S-related terminology, refer to the Section [46.2 Terminology](#) in Chapter [46 I2S Controller \(I2S\)](#).

47.2 Feature List

The LP I2S module has the following features:

- RX master mode and slave mode
- A variety of audio standards supported:
 - TDM Philips standard
 - TDM MSB alignment standard
 - TDM PCM standard
 - PDM standard
- Various RX modes supported:
 - TDM RX mode, up to 2 channels supported
 - PDM RX mode
 - * Raw PDM data reception
 - * PDM-to-PCM data format conversion, up to 2 channels supported
- Configurable sample clock with a variety of sampling frequencies supported
- 16-bit data communication

- Standard LP I2S interface interrupts

47.3 Architectural Overview

Figure 47.3-1 shows the structure of the ESP32-P4 LP I2S module, consisting of:

- Receive control unit (RX Unit)
- Input and output timing unit (I/O Sync)
- Clock divider (Clock Generator)
- 64 x 32-bit RX FIFO
- 256 x 32-bit LP I2S Memory
- Communication interface for [Voice Activity Detection \(VAD\)](#)

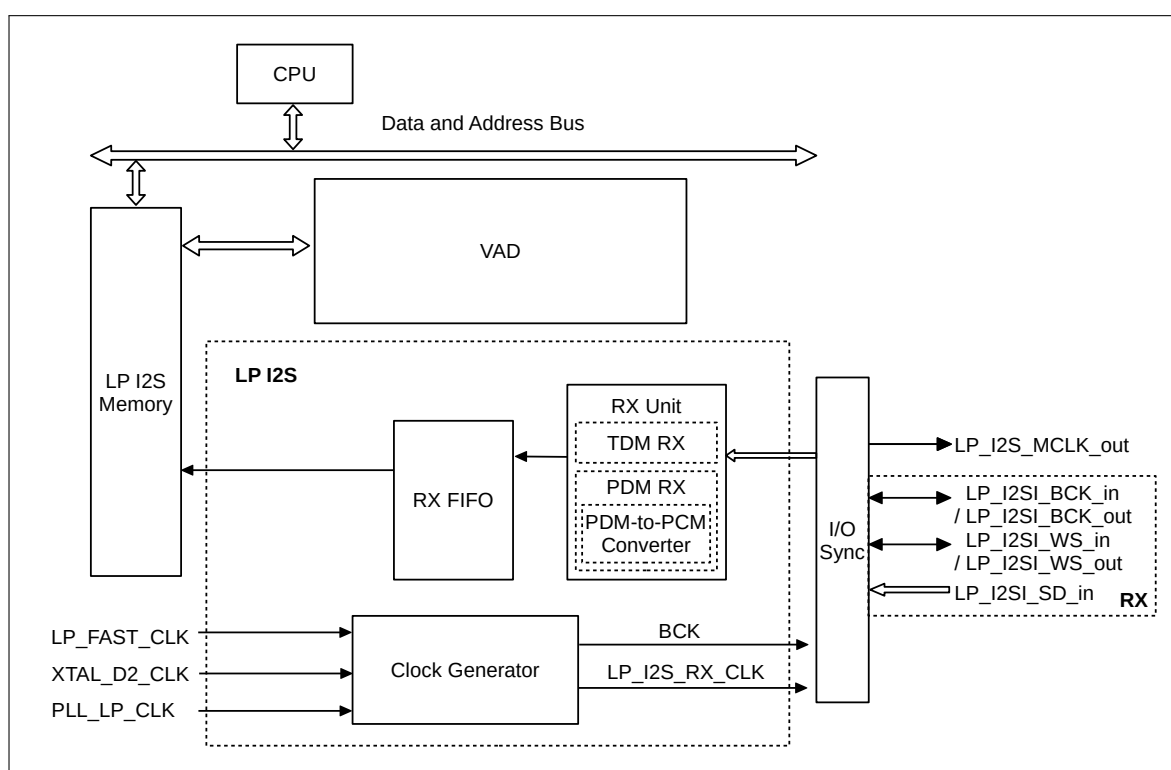


Figure 47.3-1. ESP32-P4 LP I2S Architecture

The RX unit have a three-line interface that uses a bit clock line (BCK), a word select line (WS), and a serial data line (SD). The SD line of the RX unit is dedicated for data input. BCK and WS signal lines for the RX unit can be configured as master output mode or slave input mode.

The signal bus of the LP I2S module is shown at the right part of Figure 47.3-1. The naming of these signals in RX units follows the pattern of LP_I2SA_B_C, for example, LP_I2SI_BCK_in.

- “A” is fixed to “I”, indicating that signals are input to the RX unit
- “B” represents the signal function, which includes:
 - BCK

- WS
- SD
- “C” represents the signal direction, which includes:
 - “in”: Input signal into the LP I2S module
 - “out”: Output signal from the LP I2S module

Table 47.3-1 provides a detailed description of LP I2S signals.

Table 47.3-1. LP I2S Signal Description

Signal [*]	Direction	Function
LP_I2SI_BCK_in	Input	In LP I2S slave mode, inputs BCK signal for RX unit
LP_I2SI_BCK_out	Output	In LP I2S master mode, outputs BCK signal for RX unit
LP_I2SI_WS_in	Input	In LP I2S slave mode, inputs WS signal for RX unit
LP_I2SI_WS_out	Output	In LP I2S master mode, outputs WS signal for RX unit
LP_I2SI_SD_in	Input	Serial input data bus for LP I2S RX unit
LP_I2S_MCLK_out	Output	LP I2S master clock output

^{*} Any required signals of LP I2S must be mapped to the chip's pins via LP GPIO matrix. See Chapter 9 [GPIO Matrix and IO MUX](#).

47.4 Supported Audio Standards

ESP32-P4 I2S supports multiple audio standards, including TDM Philips standard, TDM MSB alignment standard, TDM PCM standard, and PDM standard.

Select the needed standard by configuring the following bits:

- [LP_I2S_RX_TDM_EN](#)
 - 0: Disable TDM mode
 - 1: Enable TDM mode
- [LP_I2S_RX_PDM_EN](#)
 - 0: Disable PDM mode
 - 1: Enable PDM mode
- [LP_I2S_RX_MSB_SHIFT](#)
 - 0: WS and SD signals change simultaneously, i.e., enable MSB alignment standard
 - 1: WS signal changes one BCK clock cycle earlier than SD signal, i.e., enable Philips standard or select PCM standard

Note:

[LP_I2S_RX_TDM_EN](#) and [LP_I2S_RX_PDM_EN](#) must not be configured to 1 or 0 at the same time, otherwise LP I2S will transmit data incorrectly in a mode that is neither TDM nor PDM.

47.4.1 TDM Philips Standard

Philips standards require that WS signal changes one BCK clock cycle earlier than SD signal on BCK falling edge, which means that WS signal is valid from one clock cycle before transmitting the first bit of channel data and changes one clock before the end of channel data transfer. SD signal line transmits the most significant bit of audio data first.

Compared with the basic Philips standard, TDM Philips standard supports multiple channels. See Figure 47.4-1.

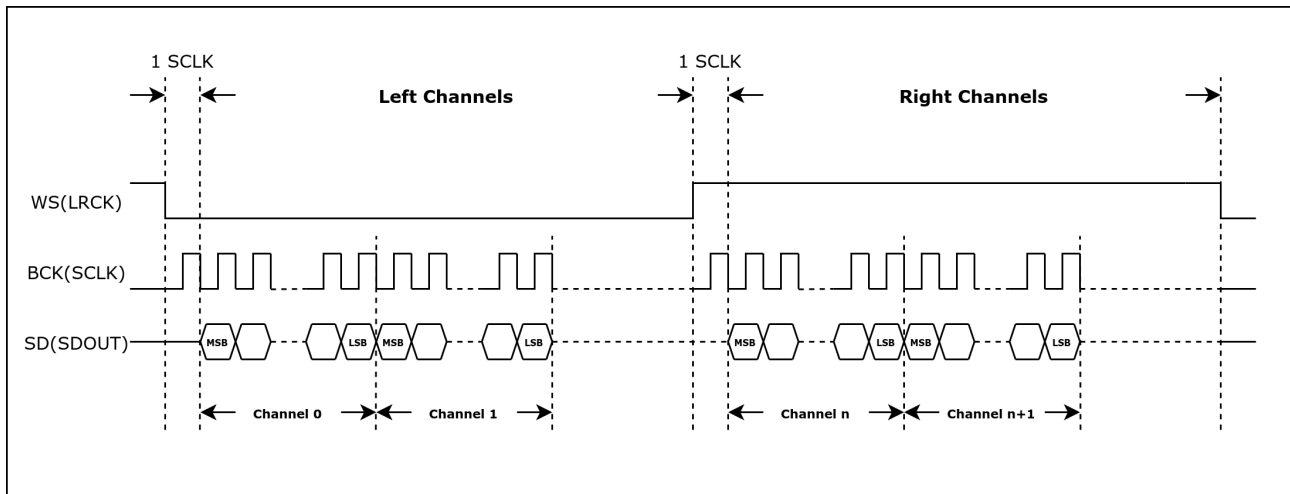


Figure 47.4-1. TDM Philips Standard Timing Diagram

47.4.2 TDM MSB Alignment Standard

MSB alignment standards require that WS and SD signals change simultaneously on the falling edge of BCK. The WS signal is valid until the end of the channel data transfer. The SD signal line transmits the most significant bit of audio data first.

Compared with the basic MSB alignment standard, TDM MSB alignment standard supports multiple channels. See Figure 47.4-2.

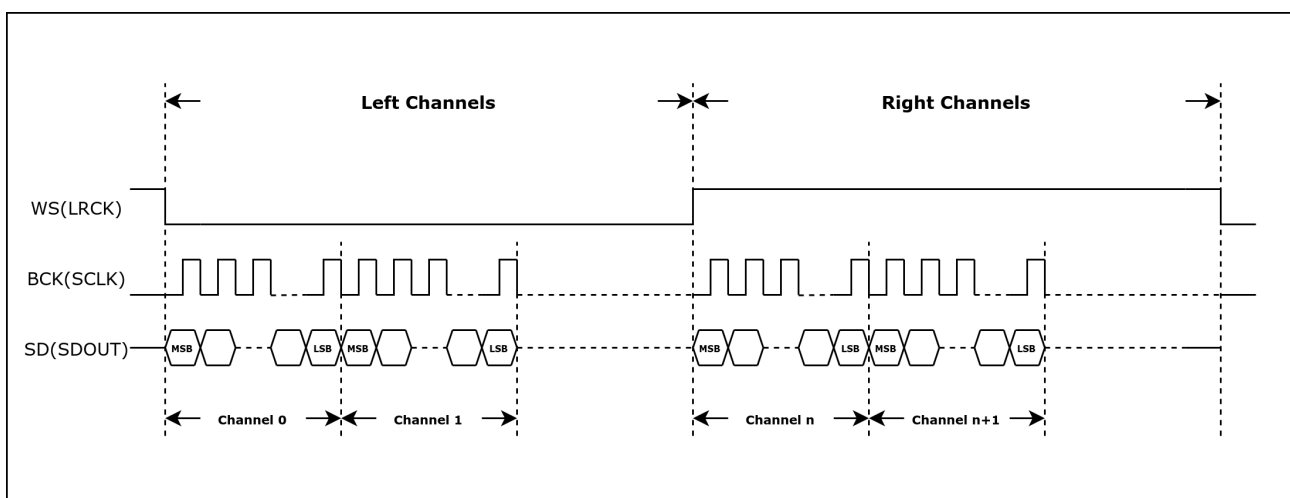


Figure 47.4-2. TDM MSB Alignment Standard Timing Diagram

47.4.3 TDM PCM Standard

Short frame synchronization under the PCM standards requires that the WS signal changes one BCK clock cycle earlier than the SD signal on the falling edge of BCK, which means that the WS signal becomes valid one clock cycle before transferring the first bit of channel data and remains unchanged in this BCK clock cycle. SD signal line first transmits the most significant bit of audio data.

Compared with the basic PCM standard, TDM PCM standard supports multiple channels. See Figure 47.4-3.

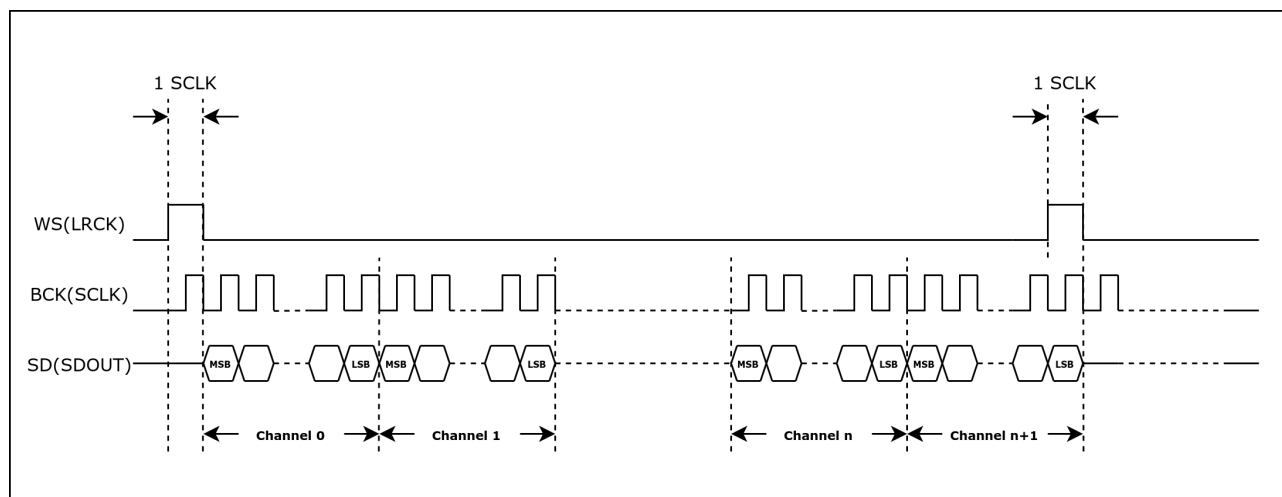


Figure 47.4-3. TDM PCM Standard Timing Diagram

47.4.4 PDM Standard

Under PDM standard, WS signal changes continuously during data transmission. The low-level and high-level of this signal indicates the left channel and right channel respectively. WS and SD signals change simultaneously on the falling edge of BCK. See Figure 47.4-4.

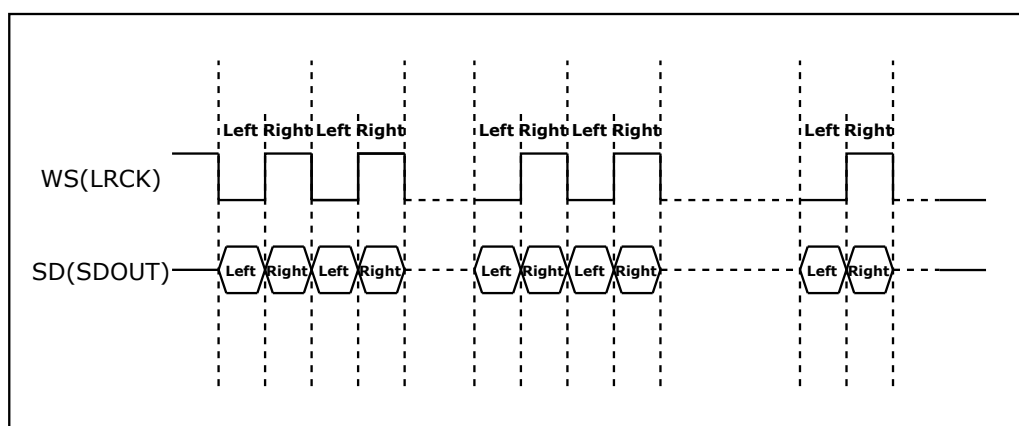


Figure 47.4-4. PDM Standard Timing Diagram

47.5 RX Clock

LP_I2S_RX_CLK is the master clock of LP I2S RX unit, divided from:

- Up to 40 MHz LP_FAST_CLK with configurable clock source
- 20 MHz XTAL_D2_CLK
- 8 MHz PLL_LP_CLK

The serial clock (BCK) of the LP I2S RX unit is divided from LP_I2S_RX_CLK, as shown in Figure 47.5-1.

[LPPERI_LP_I2S_RX_CLK_SEL](#) is used to select clock source for the LP I2S RX unit, and [LPPERI_CK_EN_LP_I2S_RX](#) to enable or disable the clock source.

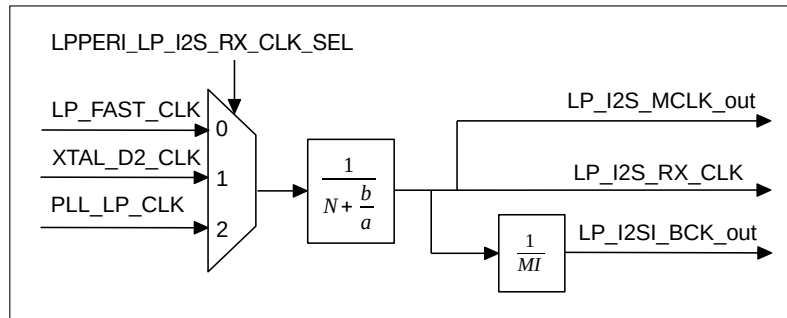


Figure 47.5-1. LP I2S Clock Generator

The following formula shows the relation between LP_I2S_RX_CLK frequency $f_{LP_I2S_RX_CLK}$ and the divider clock source frequency $f_{LP_I2S_CLK_S}$:

$$f_{LP_I2S_RX_CLK} = \frac{f_{LP_I2S_CLK_S}}{N + \frac{b}{a}}$$

N is an integer value between 2 and 256. The value of N is mapped to that of [LPPERI_LP_I2S_RX_CLKM_DIV_N](#) as follows:

- When [LPPERI_LP_I2S_RX_CLKM_DIV_N](#) = 0, N = 256;
- When [LPPERI_LP_I2S_RX_CLKM_DIV_N](#) = 1, N = 2;
- When [LPPERI_LP_I2S_RX_CLKM_DIV_N](#) has any other value, N = [LPPERI_LP_I2S_RX_CLKM_DIV_NUM](#).

The values of “a” and “b” in fractional divider depend only on x, y, z, and yn1. The corresponding formulas are as follows:

- When $b \leq \frac{a}{2}$, $yn1 = 0$, $x = \text{floor}(\lceil \frac{a}{b} \rceil) - 1$, $y = a \% b$, $z = b$;
- When $b > \frac{a}{2}$, $yn1 = 1$, $x = \text{floor}(\lceil \frac{a}{a-b} \rceil) - 1$, $y = a \% (a - b)$, $z = a - b$.

The values of x, y, z, and yn1 are configured by [LPPERI_LP_I2S_RX_CLKM_DIV_X](#), [LPPERI_LP_I2S_RX_CLKM_DIV_Y](#), [LPPERI_LP_I2S_RX_CLKM_DIV_Z](#) and [LPPERI_LP_I2S_RX_CLKM_DIV_YN1](#).

To configure the integer divider, clear [LPPERI_LP_I2S_RX_CLKM_DIV_X](#) and [LPPERI_LP_I2S_RX_CLKM_DIV_Z](#), then set [LPPERI_LP_I2S_RX_CLKM_DIV_Y](#) to 1.

Note:

Using fractional divider may introduce some clock jitter.

In master RX mode, the serial clock BCK for LP I2S RX unit is LP_I2SI_BCK_out divided from LP_I2S_RX_CLK, which is:

$$f_{\text{LP_I2S_BCK_out}} = \frac{f_{\text{LP_I2S_RX_CLK}}}{\text{MI}}$$

“MI” is an integer value:

$$\text{MI} = \text{LP_I2S_RX_BCK_DIV_NUM} + 1$$

Note:

LP_I2S_RX_BCK_DIV_NUM must not be configured as 1.

In LP I2S slave mode, make sure $f_{\text{LP_I2S_RX_CLK}} \geq 8 \times f_{\text{BCK}}$ to ensure the sampling accuracy. The LP I2S module can output LP_I2S_MCLK_out as the master clock for peripherals.

Since in slave mode, the module clock frequency must be greater than or equal to 8 times the BCK clock frequency, the maximum sampling frequency of LP I2S is limited by the data bit width and the number of channels. For example, since the clock source frequency is up to 40 MHz, the module clock can be configured up to 20 MHz, and the BCK clock can be configured up to 5 MHz. Therefore, when transmitting dual-channel 16-bit width data, LP I2S supports sample frequencies up to 78.125 kHz, e.g., 8 kHz, 16 kHz, 32 kHz, 44.1 kHz, 48 kHz. For detailed information, please refer to Section [47.5 RX Clock](#).

In addition to the clock frequency limitation, LP I2S uses LP I2S internal memory for data storage instead of DMA. Therefore, users need to read data from the LP I2S internal memory in a timely manner. The reading rate may limit the sample frequencies that LP I2S supports. For detailed information, please refer to Section [47.8.3 Internal Memory](#).

47.6 Reset

The units and FIFOs in the LP I2S module are reset by the following bits.

- LP I2S RX units: Reset by the bit [LP_I2S_RX_RESET](#);
- LP I2S RX FIFO: Reset by the bits [LP_I2S_RX_FIFO_RESET](#).

Note:

The LP I2S module clock must be configured first before the units and FIFO are reset.

47.7 Master/Slave RX Mode

The LP I2S supports receiving data either as a master or as a slave in RX mode.

- LP I2S works as a master receiver:
 - Set [LP_I2S_RX_START](#) to start receiving data. When this bit is set, the RX unit keeps outputting clock signal and sampling input data.
 - Configure [LP_I2S_RX_STOP_MODE](#) to control the suspension of data reception:
 - * 0: The RX unit only suspends data reception when [LP_I2S_RX_START](#) is cleared.
 - * 1: The RX unit suspends data reception when [LP_I2S_RX_START](#) is cleared or the number of received bytes is greater than the value configured in [LP_I2S_RX_EOF_NUM_REG](#). After data

reception is suspended, if `LP_I2S_RX_START` is not cleared, data reception can be restarted by setting `LP_I2S_RX_RESET` to 1.

- RX unit stops receiving data when the bit `LP_I2S_RX_START` is cleared.
- LP I2S works as a slave receiver:
 - Set `LP_I2S_RX_START`. Wait for the master BCK signal to start receiving data.
 - Configure `LP_I2S_RX_STOP_MODE` to control data reception suspension:
 - * 0: The RX unit only suspends data reception when `LP_I2S_RX_START` is cleared.
 - * 1: The RX unit suspends data reception when `LP_I2S_RX_START` is cleared or the number of received bytes is greater than the value configured in `LP_I2S_RX_EOF_NUM_REG`. After data reception is suspended, if `LP_I2S_RX_START` is not cleared, data reception can be restarted by setting `LP_I2S_RX_RESET` to 1.
 - The RX unit stops receiving data when the bit `LP_I2S_RX_START` is cleared.

47.8 Receiving Data

In RX mode, LP I2S first reads data from the peripheral interface and then stores the data in the LP I2S memory according to the configured channel mode and data mode.

47.8.1 Channel Mode Control

ESP32-P4 LP I2S supports both TDM RX mode and PDM RX mode. Set `LP_I2S_RX_TDM_EN` to enable TDM RX mode, or set `LP_I2S_RX_PDM_EN` to enable PDM RX mode.

Note:

`LP_I2S_RX_TDM_EN` and `LP_I2S_RX_PDM_EN` must not be cleared or set simultaneously.

47.8.1.1 TDM RX Mode

In TDM RX mode, LP I2S supports up to 2 channels to input data. The total number of RX channels in use is controlled by `LP_I2S_RX_TDM_TOT_CHAN_NUM`. For example, if `LP_I2S_RX_TDM_TOT_CHAN_NUM` is set to 1, channel 0 ~ 1 will be used to receive data.

In these RX channels, if `LP_I2S_RX_TDM_PDM_CHANn_EN` is set to:

- 0: The channel data is invalid and will not be stored into RX FIFO;
- 1: The channel data is valid and will be stored into RX FIFO.

In TDM master mode, WS signal is controlled by `LP_I2S_RX_WS_IDLE_POL` and `LP_I2S_RX_TDM_WS_WIDTH`.

- `LP_I2S_RX_WS_IDLE_POL`: The default level of WS signal;
- `LP_I2S_RX_TDM_WS_WIDTH`: The cycles the WS default level lasts for when receiving all channel data.

`LP_I2S_RX_HALF_SAMPLE_BITS` x 2 is equal to the BCK cycles in one WS period.

47.8.1.2 PDM RX Mode

In PDM RX mode, LP I2S converts the serial data from channels to the data to be entered into memory. LP I2S supports both PDM raw data reception and PDM-to-PCM data format conversion.

In PDM RX master mode, the default level of the WS signal is controlled by [LP_I2S_RX_WS_IDLE_POL](#). WS frequency is half of the BCK frequency. The configuration of the BCK signal is similar to that of WS signal as described in Section 47.5. Note, in PDM RX mode, the value of [LP_I2S_RX_HALF_SAMPLE_BITS](#) must be same as that of [LP_I2S_RX_BITS_MOD](#).

When the PDM-to-PCM converter is enabled for LP I2S, the received PDM data is converted to PCM data and controlled according to the data mode. Configure [LP_I2S_RX_PDM2PCM_EN](#) to enable this converter. The register configuration for PDM-to-PCM converter is as follows:

- Configure sampling frequency and downsampling rate.

When LP I2S PDM-to-PCM converter is enabled, PDM clock frequency is:

- in master mode: PDM clock frequency is equal to BCK frequency.
- in slave mode: PDM clock is provided by external device.

The sampling frequency (f_{Sampling}) is related to PDM clock frequency as follows:

$$f_{\text{Sampling}} = \frac{f_{\text{PDM}}}{\text{DSR}}$$

Downsampling rate (DSR) is related to [LP_I2S_RX_PDM_SINC_DSR_16_EN](#) as follows:

$$\text{DSR} = \text{LP_I2S_RX_PDM_SINC_DSR_16_EN} \times 64$$

Configure the registers according to needed master/slave mode, sampling frequency, and downsampling rate.

- Configure valid channels.

When the PDM-to-PCM converter is enabled, input signals from eight channels are supported at most. See Table 47.8-1 for the register configuration and related channels.

Table 47.8-1. PDM-to-PCM Data Input

Input Data Signal	Channel	Enable Register
LP_I2SI_SD_in	Left channel	LP_I2S_RX_TDM_PDM_CHANO_EN
	Right channel	LP_I2S_RX_TDM_PDM_CHAN1_EN

47.8.2 Data Format Control

The data format of LP I2S is controlled in the following phases:

- Phase I: Serial input data is converted into the data to be saved to RX FIFO;
- Phase II: The data is read from RX FIFO and converted according to the input data mode.

47.8.2.1 Bit Order Control of Channel Data

The channel data will be stored as the data to be input in order from high to low. The data bit order in each channel is controlled by [LP_I2S_RX_BIT_ORDER](#):

- 0: The bit order of the data to be input is not reversed;
- 1: The bit order of the data to be input is reversed.

At this point, the first phase of data format control is completed.

47.8.2.2 Bit Width Control of Channel RX Data

The storage data width in each channel is controlled by [LP_I2S_RX_BITS_MOD](#). For ESP32-P4 LP I2S, the value of [LP_I2S_RX_BITS_MOD](#) is fixed as 15. The corresponding channel storage data width is 16 bit.

47.8.2.3 Bit Width Control of Channel RX Data

The RX data width in each channel is determined by [LP_I2S_RX_TDM_CHAN_BITS](#).

- If the storage data width in each channel is smaller than the received (RX) data width, then only the bits within the storage data width is saved into memory. Configure [LP_I2S_RX_LEFT_ALIGN](#) to:
 - 0: Only the lower bits of the received data within the storage data width is stored to memory;
 - 1: Only the higher bits of the received data within the storage data width is stored to memory.
- If the received data width is smaller than the storage data width in each channel, the higher bits of the received data will be filled with zeros and then the data is saved to memory.

47.8.2.4 Endian Control of Channel Storage Data

The received data is then converted into storage data (to be stored to memory) after some processing, such as discarding extra bits or filling zeros in missing bits. The endian of the storage data is controlled by [LP_I2S_RX_BIG_ENDIAN](#) (the data bit width is fixed as 16). See the table below.

Table 47.8-2. Channel Storage Data Endian

Original Data	Endian of Processed Data	LP_I2S_RX_BIG_ENDIAN
{B1, B0}	{B1, B0}	0
	{B0, B1}	1

At this point, the data format control is completed. Data then is stored into memory.

47.8.3 Internal Memory

All data after format control is stored in the LP I2S internal memory, which is a 16-bit wide circular buffer that supports RX unit write operations and system read operations. The LP I2S internal memory includes a pair of hardware-maintained read and write pointers to manage write and read operations.

1. When LP I2S writes data:

- If the memory is full: Overwrites the oldest data and increments the write and read pointers by 1 (the value of the read pointer is the value of the write pointer plus one at this point).
- If the memory is not full: Writes the data and increments the write pointer by 1. The read pointer remains unchanged.

2. When the system reads from the LP I2S memory:

- If the memory is empty: The read and write pointers remain unchanged (the value of the read pointer equals to the value of the write pointer at this point).
- If the memory is not empty: Increments the read pointer by 1. The write pointer remains unchanged.

Note:

- The incremental operation for the read and write pointers is in a circular manner, i.e., when the value of the read or write pointer reaches the boundary of the memory, the incremental operation resets the value of the pointer to zero.
- Within the LP I2S internal memory architecture, the system always reads the oldest data in the memory.

47.9 Interrupts

ESP32-P4's LP I2S can generate the LP_I2S_INTR interrupt signal that will be sent to the [Interrupt Matrix](#). There are several internal interrupt sources from LP I2S that can generate the above interrupt signals as follows:

- LP_I2S_RX_HUNG_INT: Triggered when the data reception is timed out. For example, if the LP I2S module is configured as RX slave mode, but the master does not transmit data for a long time (specified in [LP_I2S_LC_HUNG_CONF_REG](#)), this interrupt will be triggered.
- LP_I2S_RX_DONE_INT: Triggered when the data reception is completed.
- LP_I2S_RX_FIFOMEM_UDF_INT: Triggered when the LP I2S memory is read empty.
- LP_I2S_RX_MEM_THRESHOLD_INT: Triggered when the data in LP I2S memory is larger than the value configured in [LP_I2S_RX_MEM_THRESHOLD](#).

Note:

For definitions of *interrupt*, *interrupt signal*, *interrupt source*, and their correlations, please refer to Chapter [12 Interrupt Matrix](#) > Section [12.2 Interrupt Terminology in ESP32-P4](#).

Each interrupt source can be configured by a common set of registers that are described in Section [Interrupt Configuration Registers](#). The specific registers can be found in Section [47.10 Register Summary](#).

47.10 Register Summary

The addresses in this section are relative to LP I2S base address provided in Table 7.3-2 in Chapter 7 *System and Memory*.

The abbreviations given in Column **Access** are explained in Section *Access Types for Registers*.

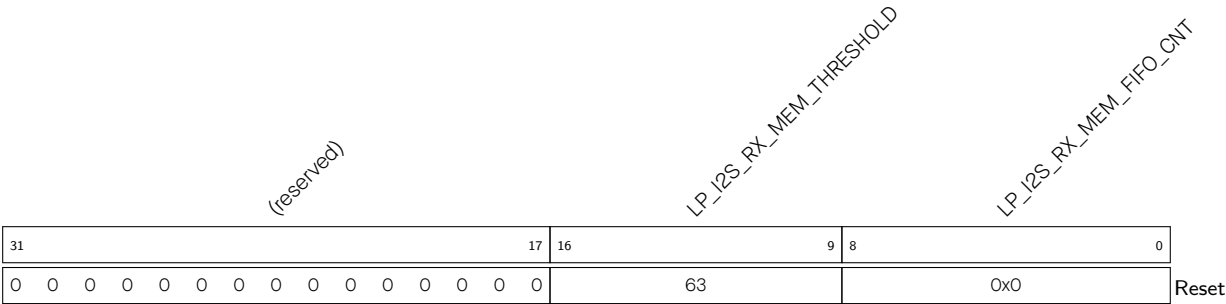
Name	Description	Address	Access
RX control and configuration registers			
LP_I2S_RX_MEM_CONF_REG	LP I2S memory configuration register	0x0008	varies
LP_I2S_RX_CONF_REG	LP I2S RX configuration register	0x0020	varies
LP_I2S_RX_CONF1_REG	LP I2S RX configuration register 1	0x0028	R/W
LP_I2S_RX_TDM_CTRL_REG	LP I2S RX TDM mode configuration register	0x0050	R/W
LP_I2S_RXEOF_NUM_REG	LP I2S RX data number control register	0x0064	R/W
LP_I2S_RX_PDM_CONF_REG	LP I2S RX PDM mode configuration register	0x0070	R/W
Interrupt registers			
LP_I2S_INT_RAW_REG	LP I2S interrupt raw register	0x000C	RO/WTC/SS
LP_I2S_INT_ST_REG	LP I2S interrupt status register	0x0010	RO
LP_I2S_INT_ENA_REG	LP I2S interrupt enable register	0x0014	R/W
LP_I2S_INT_CLR_REG	LP I2S interrupt clear register	0x0018	WT
RX clock and timing register			
LP_I2S_RX_TIMING_REG	LP I2S RX timing control register	0x0058	R/W
Control and configuration registers			
LP_I2S_LC_HUNG_CONF_REG	LP I2S timeout configuration register	0x0060	R/W
LP_I2S_CONF_SIGLE_DATA_REG	LP I2S single data register	0x0068	R/W
Clock register			
LP_I2S_CLK_GATE_REG	Clock gate register	0x00F8	R/W
Version register			
LP_I2S_DATE_REG	Version control register	0x00FC	R/W

47.11 Registers

The addresses in this section are relative to LP I2S base address provided in Table 7.3-2 in Chapter 7 *System and Memory*.

For how to program reserved fields, please refer to Section *Programming Reserved Register Field*.

Register 47.1. LP_I2S_RX_MEM_CONF_REG (0x0008)



- LP_I2S_RX_MEM_FIFO_CNT Configures the number of data in the RX memory. (RO)
- LP_I2S_RX_MEM_THRESHOLD Configures the RX memory threshold. LP I2S RX memory triggers an interrupt when the data in the memory exceeds (not including equals) this threshold. (R/W)

Register 47.2. LP_I2S_RX_CONF_REG (0x0020)

[illegible]

LP_I2S_RX_RESET Configures whether to reset RX.

0: No effect

1: Reset

(WT)

LP_I2S_RX_FIFO_RESET Configures whether to reset RX FIFO.

0: No effect

1: Reset

(WT)

LP_I2S_RX_START Configures whether to start receiving data.

0: No effect

1: Start

(R/W)

Continued on the next page...

Register 47.2. LP_I2S_RX_CONF_REG (0x0020)

Continued from the previous page...

LP_I2S_RX_SLAVE_MOD Configures whether to enable slave RX mode.

0: Disable

1: Enable

(R/W)

LP_I2S_RX_FIFOMEM_RESET Configures whether to reset RX Sync FIFO memory.

0: No effect

1: Reset

(WT)

LP_I2S_RX_MONO Configures whether to enable RX unit in mono mode.

0: Disable

1: Enable

(R/W)

LP_I2S_RX_BIG_ENDIAN Configures LP I2S RX byte endian.

0: Low address data is saved to low address

1: Low address data is saved to high address

(R/W)

LP_I2S_RX_UPDATE Configures whether to update LP I2S RX registers from APB clock domain to LP I2S RX clock domain. This bit will be cleared by hardware after the register update is done.

0: No effect

1: Update

(R/W/SC)

LP_I2S_RX_MONO_FST_VLD Configures which channel data value is valid in LP I2S RX mono mode.

0: The second channel data value is valid

1: The first channel data value is valid

(R/W)

LP_I2S_RX_STOP_MODE Configures when LP I2S RX stops data reception.

0: Only stops when [LP_I2S_RX_START](#) is cleared

1: Stops when [LP_I2S_RX_START](#) is cleared or the number of received bytes is greater than the value configured in [LP_I2S_RX_EOF_NUM_REG](#)

Other values: Invalid

(R/W)

LP_I2S_RX_LEFT_ALIGN Configures the RX alignment mode

0: Right alignment

1: Left alignment

(R/W)

Continued on the next page...

Register 47.2. LP_I2S_RX_CONF_REG (0x0020)

Continued from the previous page...

LP_I2S_RX_WS_IDLE_POL Configures the relationship between WS level and which channel data to receive.

0: WS remains low when receiving left channel data and high when receiving right channel data

1: WS remains high when receiving left channel data and low when receiving right channel data

(R/W)

LP_I2S_RX_BIT_ORDER Configures whether to reverse the bit order of the LP I2S RX data to be received.

0: Not reverse

1: Reverse

(R/W)

LP_I2S_RX_TDM_EN Configures whether to enable LP I2S TDM RX mode.

0: Disable

1: Enable

(R/W)

LP_I2S_RX_PDM_EN Configures whether to enable LP I2S PDM RX mode.

0: Disable

1: Enable

(R/W)

Register 47.3. LP_I2S_RX_CONF1_REG (0x0028)

(reserved)		LP_I2S_RX_MSB_SHIFT		LP_I2S_RX_TDM_CHAN_BITS		LP_I2S_RX_HALF_SAMPLE_BITS		LP_I2S_RX_BITS_MOD		LP_I2S_RX_BCK_DIV_NUM		LP_I2S_RX_TDM_WS_WIDTH	
31	30	29	28	24	23	18	17	13	12	7	6		0
0	0	1		0xf		0xf		0xf		6		0x0	Reset

LP_I2S_RX_TDM_WS_WIDTH Configures the width of LP_I2SI_WS_out (WS default level) at idle level in TDM mode. Width of LP_I2SI_WS_out at idle level in TDM mode = $(LP_I2S_RX_TDM_WS_WIDTH[6:0] + 1) \times T_BCK$. (R/W)

LP_I2S_RX_BCK_DIV_NUM Configures the divider of BCK in RX mode. Note this divider must not be configured to 1. (R/W)

LP_I2S_RX_BITS_MOD Configures the valid data bit length of LP I2S RX channel. Since the LP I2S only supports 16-bit mode, this field must be configured to 15. LP I2S cannot function properly if this field is configured to any other value. (R/W)

LP_I2S_RX_HALF_SAMPLE_BITS Configures LP I2S RX sample bits. BCK cycles in one WS period = $I2S_RX_HALF_SAMPLE_BITS \times 2$. (R/W)

LP_I2S_RX_TDM_CHAN_BITS Configures RX bit number for each channel in TDM mode. Bit number expected = $LP_I2S_RX_TDM_CHAN_BITS + 1$. (R/W)

LP_I2S_RX_MSB_SHIFT Configures the timing between the WS signal and the MSB of data.

0: Align at the rising edge

1: The WS signal changes one BCK clock earlier

(R/W)

Register 47.4. LP_I2S_RX_TDM_CTRL_REG (0x0050)

(reserved)											LP_I2S_RX_TDM_TOT_CHAN_NUM					(reserved)										LP_I2S_RX_TDM_PDM_CHAN1_EN		LP_I2S_RX_TDM_PDM_CHAN0_EN			
31											20	19	16			15											2	1	0		
0 0 0 0 0 0 0 0 0 0											0x0			0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0										1	1	Reset					

LP_I2S_RX_TDM_PDM_CHAN0_EN Configures whether to enable the valid data input of LP I2S RX TDM or PDM channel 0.

0: Disable. Channel 0 only inputs 0

1: Enable

(R/W)

LP_I2S_RX_TDM_PDM_CHAN1_EN Configures whether to enable the valid data input of LP I2S RX TDM or PDM channel 1.

0: Disable. Channel 1 only inputs 0

1: Enable

(R/W)

LP_I2S_RX_TDM_TOT_CHAN_NUM Configures the total channel number of LP I2S RX TDM mode.

Total channel number in use = LP_I2S_RX_TDM_TOT_CHAN_NUM + 1. (R/W)

Register 47.5. LP_I2S_RXEOF_NUM_REG (0x0064)

(reserved)																LP_I2S_RX_EOF_NUM																																															
31																12																11																0															
0 0																0x40																Reset																															

LP_I2S_RX_EOF_NUM Configures the bit length of RX data. Bit length of RX data = (LP_I2S_RX_BITS_MOD[4:0] + 1) x (LP_I2S_RX_EOF_NUM[11:0] + 1). Valid when **LP_I2S_RX_STOP_MODE** is set to 1. (R/W)

Register 47.6. LP_I2S_RX_PDM_CONF_REG (0x0070)

(reserved)				(reserved)		LP_I2S_RX_PDM_HP_BYPASS		LP_I2S_RX_PDM2PCM_AMPLIFY_NUM		LP_I2S_RX_PDM_SINC_DSR_16_EN		(reserved)																			
31	29	28	26	25	24	21	20	19	18																						0
0x7				0x6		0		0x1		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	Reset

LP_I2S_RX_PDM2PCM_EN Configures whether to enable the PDM-to-PCM converter in RX mode.

0: Disable

1: Enable

(R/W)

LP_I2S_RX_PDM_SINC_DSR_16_EN Configures the downsampling rate of PDM RX filter group 1 module.

0: 64

1: 128

(R/W)

LP_I2S_RX_PDM2PCM_AMPLIFY_NUM Configures the PDM-to-PCM RX amplification coefficient.

PCM data will be multiplied by this value before outputting. (R/W)

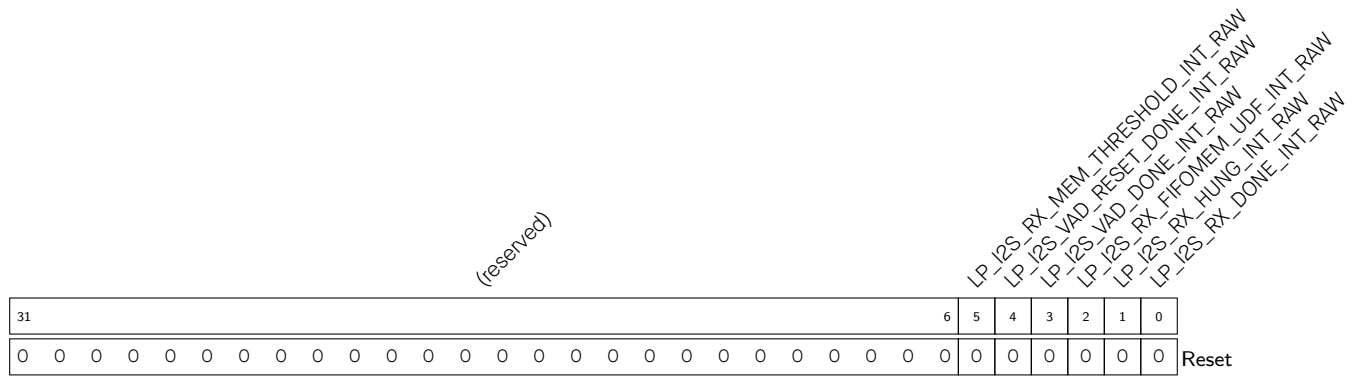
LP_I2S_RX_PDM_HP_BYPASS Configures whether PDM-to-PCM RX bypasses the HP filter.

0: Not bypass

1: Bypass

(R/W)

Register 47.7. LP_I2S_INT_RAW_REG (0x000C)



LP_I2S_RX_DONE_INT_RAW The raw interrupt status of the [LP_I2S_RX_DONE_INT](#) interrupt. (R/SS/WTC)

LP_I2S_RX_HUNG_INT_RAW The raw interrupt status of the [LP_I2S_RX_DONE_INT](#) interrupt. (R/SS/WTC)

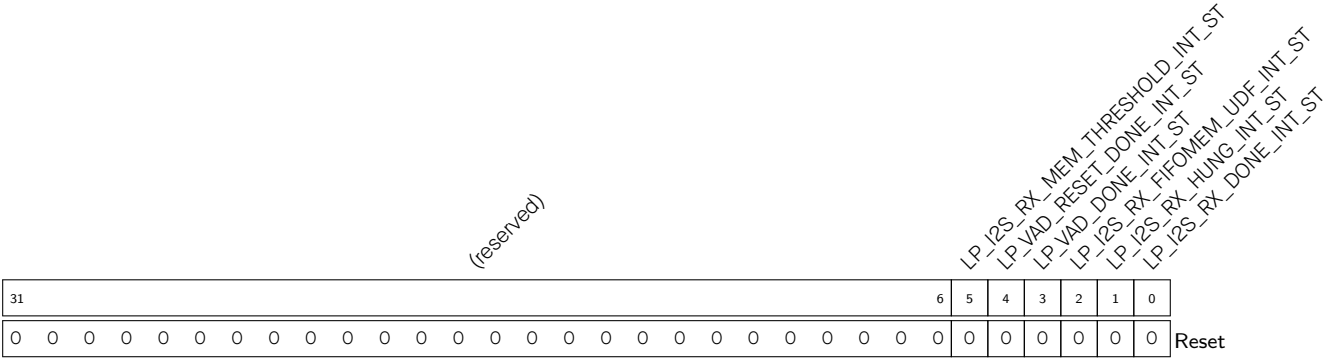
LP_I2S_RX_FIFOMEM_UDF_INT_RAW The raw interrupt status of the [LP_I2S_RX_FIFOMEM_UDF_INT](#) interrupt. (R/SS/WTC)

LP_I2S_VAD_DONE_INT_RAW The raw interrupt status of the [LP_I2S_VAD_DONE_INT](#) interrupt. (R/SS/WTC)

LP_I2S_VAD_RESET_DONE_INT_RAW The raw interrupt status of the [LP_I2S_VAD_RESET_DONE_INT](#) interrupt. (R/SS/WTC)

LP_I2S_RX_MEM_THRESHOLD_INT_RAW The raw interrupt status of the [LP_I2S_RX_MEM_THRESHOLD_INT](#) interrupt. (R/SS/WTC)

Register 47.8. LP_I2S_INT_ST_REG (0x0010)



- LP_I2S_RX_DONE_INT_ST

The masked interrupt status of the [LP_I2S_RX_DONE_INT](#) interrupt. (RO)
- LP_I2S_RX_HUNG_INT_ST

The masked interrupt status of the [LP_I2S_RX_DONE_INT](#) interrupt. (RO)
- LP_I2S_RX_FIFOMEM_UDF_INT_ST

The masked interrupt status of the [LP_I2S_RX_FIFOMEM_UDF_INT](#) interrupt. (RO)
- LP_I2S_VAD_DONE_INT_ST

The masked interrupt status of the [LP_I2S_VAD_DONE_INT](#) interrupt. (RO)
- LP_I2S_VAD_RESET_DONE_INT_ST

The masked interrupt status of the [LP_I2S_VAD_RESET_DONE_INT](#) interrupt. (RO)
- LP_I2S_RX_MEM_THRESHOLD_INT_ST

The masked interrupt status of the [LP_I2S_RX_MEM_THRESHOLD_INT](#) interrupt. (RO)

Register 47.9. LP_I2S_INT_ENA_REG (0x0014)

(reserved)																										LP_I2S_RX_MEM_THRESHOLD_INT_ENA LP_VAD_RESET_DONE_INT_ENA LP_VAD_DONE_INT_ENA LP_I2S_RX_FIFOMEM_UDF_INT_ENA LP_I2S_RX_HUNG_INT_ENA LP_I2S_RX_DONE_INT_ENA																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																		
31																										6	5	4	3	2	1	0	Reset																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																											
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

- LP_I2S_RX_DONE_INT_ENA Write 1 to enable the LP_I2S_RX_DONE_INT interrupt. (R/W)
- LP_I2S_RX_HUNG_INT_ENA Write 1 to enable the LP_I2S_RX_DONE_INT interrupt. (R/W)
- LP_I2S_RX_FIFOMEM_UDF_INT_ENA Write 1 to enable the LP_I2S_RX_FIFOMEM_UDF_INT interrupt. (R/W)
- LP_VAD_DONE_INT_ENA Write 1 to enable the LP_I2S_VAD_DONE_INT interrupt. (R/W)
- LP_VAD_RESET_DONE_INT_ENA Write 1 to enable the LP_I2S_VAD_RESET_DONE_INT interrupt. (R/W)
- LP_I2S_RX_MEM_THRESHOLD_INT_ENA Write 1 to enable the LP_I2S_RX_MEM_THRESHOLD_INT interrupt. (R/W)

Register 47.10. LP_I2S_INT_CLR_REG (0x0018)

Diagram of the LP_I2S_RX_DONE_INT_CLR register structure:

- Bit 31: (reserved)
- Bit 30: (reserved)
- Bit 29: (reserved)
- Bit 28: (reserved)
- Bit 27: (reserved)
- Bit 26: (reserved)
- Bit 25: (reserved)
- Bit 24: (reserved)
- Bit 23: (reserved)
- Bit 22: (reserved)
- Bit 21: (reserved)
- Bit 20: (reserved)
- Bit 19: (reserved)
- Bit 18: (reserved)
- Bit 17: (reserved)
- Bit 16: (reserved)
- Bit 15: (reserved)
- Bit 14: (reserved)
- Bit 13: (reserved)
- Bit 12: (reserved)
- Bit 11: (reserved)
- Bit 10: (reserved)
- Bit 9: (reserved)
- Bit 8: (reserved)
- Bit 7: (reserved)
- Bit 6: (reserved)
- Bit 5: LP_I2S_RX_MEM_THRESHOLD_INT_CLR
- Bit 4: LP_I2S_RX_DONE_INT_CLR
- Bit 3: LP_I2S_RX_FIFOEMEM_UDF_INT_CLR
- Bit 2: LP_I2S_RX_HUNG_INT_CLR
- Bit 1: LP_I2S_RX_DONE_INT_CLR
- Bit 0: LP_I2S_RX_DONE_INT_CLR

Reset

LP_I2S_RX_DONE_INT_CLR Write 1 to clear the **LP_I2S_RX_DONE_INT** interrupt. (WT)

LP_I2S_RX_HUNG_INT_CLR Write 1 to clear the **LP_I2S_RX_DONE_INT** interrupt. (WT)

LP_I2S_RX_FIFOMEM_UDF_INT_CLR Write 1 to clear the LP_I2S_RX_FIFOMEM_UDF_INT interrupt.
(WT)

LP_VAD_DONE_INT_CLR Write 1 to clear the **LP_I2S_VAD_DONE_INT** interrupt. (WT)

LP_VAD_RESET_DONE_INT_CLR Write 1 to clear the [LP_I2S_VAD_RESET_DONE_INT](#) interrupt. (WT)

LP_I2S_RX_MEM_THRESHOLD_INT_CLR Write 1 to clear the [LP_I2S_RX_MEM_THRESHOLD_INT](#) interrupt. (WT)

Register 47.11. LP_I2S_RX_TIMING_REG (0x0058)

(reserved)		LP_I2S_RX_BCK_IN_DM		(reserved)		LP_I2S_RX_WS_IN_DM		(reserved)		LP_I2S_RX_BCK_OUT_DM		(reserved)		LP_I2S_RX_WS_OUT_DM		(reserved)										LP_I2S_RX_SD_IN_DM					
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16											2	1	0			
0	0	0x0		0	0	0x0		0	0	0x0		0	0	0x0		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0x0	Reset

Reset

LP_I2S_RX_SD_IN_DM Configures the delay mode of LP I2S RX SD input signal.

0: Bypass

1: Delay by positive edge

2: Delay by negative edge

3: Invalid

(R/W)

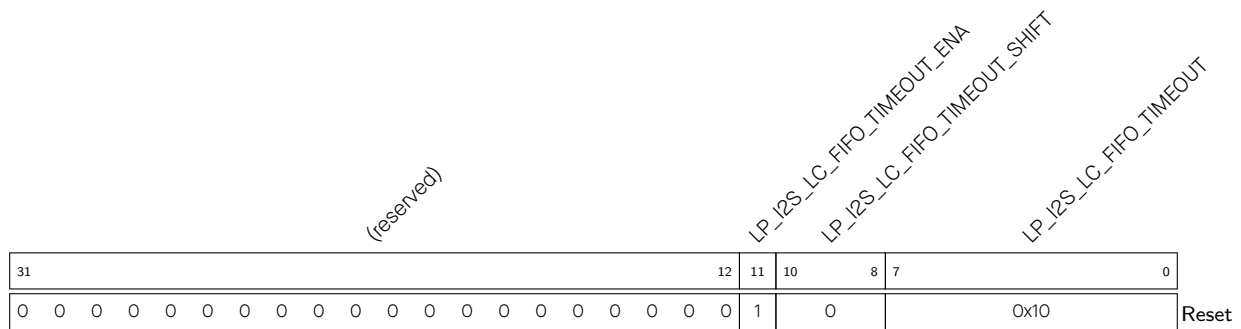
LP_I2S_RX_WS_OUT_DM Configures the delay mode of LP I2S RX WS output signal. For detailed configuration values, please refer to [LP_I2S_RX_SD_IN_DM](#). (R/W)

LP_I2S_RX_BCK_OUT_DM Configures the delay mode of LP I2S RX BCK output signal. For detailed configuration values, please refer to [LP_I2S_RX_SD_IN_DM](#). (R/W)

LP_I2S_RX_WS_IN_DM Configures the delay mode of LP I2S RX WS input signal. For detailed configuration values, please refer to [LP_I2S_RX_SD_IN_DM](#). (R/W)

LP_I2S_RX_BCK_IN_DM Configures the delay mode of LP I2S RX BCK input signal. For detailed configuration values, please refer to [LP_I2S_RX_SD_IN_DM](#). (R/W)

Register 47.12. LP_I2S_LC_HUNG_CONF_REG (0x0060)

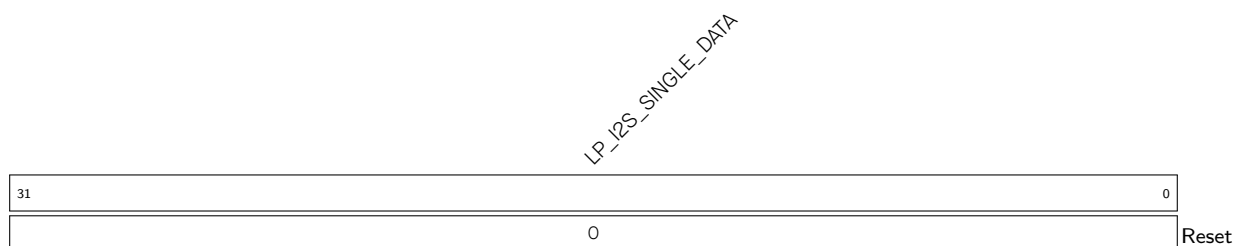


LP_I2S_LC_FIFO_TIMEOUT Configures FIFO timeout threshold. The FIFO hung counter is incremented by one each time LP I2S is in an error state and the tick counter reaches its threshold. [LP_I2S_RX_HUNG_INT](#) interrupt will be triggered when FIFO hung counter is equal to the FIFO timeout threshold. (R/W)

LP_I2S_LC_FIFO_TIMEOUT_SHIFT Configures tick counter threshold. The tick counter is incremented by one each time LP I2S is in an error state and it is on the rising edge in each system clock (APB). The tick counter is reset when counter value $\geq 88000/2^{LP_I2S_LC_FIFO_TIMEOUT_SHIFT}$. (R/W)

LP_I2S_LC_FIFO_TIMEOUT_ENA The enable bit for FIFO timeout. Configures whether to enable FIFO timeout.
 0: Disable
 1: Enable
 (R/W)

Register 47.13. LP_I2S_CONF_SIGLE_DATA_REG (0x0068)



LP_I2S_SINGLE_DATA Configures the constant channel data to be sent out. (R/W)

Register 47.14. LP_I2S_CLK_GATE_REG (0x00F8)

(reserved)																												LP_I2S_RX_LP_I2S_CG_FORCE_ON LP_I2S_RX_LP_I2S_CG_FORCE_ON LP_I2S_RX_LP_I2S_CG_FORCE_ON LP_I2S_RX_LP_I2S_CG_FORCE_ON				
31																												4	3	2	1	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1	0	Reset

LP_I2S_CLK_EN Configures whether to enable clock gate.

0: Disable

1: Enable

(R/W)

LP_I2S_VAD_CG_FORCE_ON Configures whether to force the VAD clock gate on.

0: No effect

1: Force on

(R/W)

LP_I2S_RX_MEM_CG_FORCE_ON Configures whether to force the LP I2S RX memory clock gate on.

0: No effect

1: Force on

(R/W)

LP_I2S_RX_LP_I2S_CG_FORCE_ON Configures whether to force the LP I2S RX register clock gate on.

0: No effect

1: Force on

(R/W)

Register 47.15. LP_I2S_DATE_REG (0x00FC)

(reserved)				LP_I2S_DATE																							31	28	27					0	
0	0	0	0	0x2305040																															Reset

LP_I2S_DATE Version control register. (R/W)